Manual de ejecución | Grupo A

1. Preparar un entorno separado

Descomprimir el ZIP en una carpeta nueva. Conservar data/source sin modificaciones. Instalar Python 3.12, Java 21, PostgreSQL 17.6, NiFi 2.12.0 y PostgreSQL JDBC 42.7.7 desde sus fuentes oficiales. Se recomienda 8 GB de RAM o más para el laboratorio y espacio adicional para motores y Spark.

```
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r 02_Codigo/requirements.txt
.venv/Scripts/Activate.ps1
```

Activar el entorno antes de ejecutar los comandos que comienzan con python. Como alternativa, sustituir python por .venv/Scripts/python.exe en cada invocación.

2. Variables para la sesión PowerShell

```
$env:JAVA_HOME = "C:/ruta/jdk-21"
$env:NIFI_RUNTIME = "C:/ruta/nifi-2.12.0"
$env:DATABASE_URL = "postgresql://lcd301:lcd301_local@127.0.0.1:55432/lcd301"
```

Crear un usuario/base de laboratorio lcd301 en PostgreSQL y escuchar solo en localhost:55432. No usar la contraseña de ejemplo en un servicio público. Arrancar NiFi con su distribución oficial; el cliente obtiene la identidad local generada del log o utiliza NIFI_USERNAME/NIFI_PASSWORD si están configurados.

3. Iniciar las aplicaciones

```
./02_Codigo/iniciar_aplicaciones.ps1 -Python .venv/Scripts/python.exe
```

Servicio	Dirección
API y documentación	http://127.0.0.1:18080/docs
Salud fuente / API	http://127.0.0.1:18080/health
Dashboard	http://127.0.0.1:18050
NiFi	https://localhost:8443/nifi/
PostgreSQL	127.0.0.1:55432 · base lcd301

La versión Docker se describe en 02_Codigo/deploy/README.md. Se suministra con validación estática; no fue ejecutada aquí. Los resultados verificados provienen del laboratorio nativo.

Ejecutar BASE y el lote incremental

4. Crear un flujo propio

nifi_ids.json identifica componentes de una instancia concreta. El constructor comprueba el grupo y reconstruye uno propio si los IDs no existen en la nueva instalación; conserva la evidencia anterior y no modifica grupos ajenos. Crea un grupo independiente y exporta su configuración.

```
python 02_Codigo/ingesta/construir_flujo.py --project-root .
```

5. Ingerir y sellar BASE

```
python 02_Codigo/ingesta/ejecutar_lote.py --project-root . --batch-id base
```

El controlador copia el catálogo a inbox, ejecuta la lectura REST, espera que las colas terminen y escribe data/manifests/base.json. Si falla o vence la espera, revisar evidencia y cuarentena; no fabricar el manifiesto ni ejecutar Spark con datos parciales.

6. Procesar un lote cerrado

```
python 02_Codigo/procesamiento/job.py --project-root . --batch-id base
--events-dir data/landing/events/base
--jdbc-jar C:/ruta/postgresql-42.7.7.jar
--report data/evidence/initial_run.json
```

Las líneas del ejemplo representan una sola invocación; unirlas o usar el carácter de continuación de PowerShell. DATABASE_URL configura la conexión. Abrir el panel y esperar su actualización; comprobar ingreso neto y pedidos pagados.

7. Repetición y DELTA

Repetir el job BASE sobre los mismos archivos debe publicar cero hechos nuevos. Después ejecutar el controlador con --batch-id delta y el job con --batch-id delta --events-dir data/landing/events/delta. Se esperan siete hechos y +280,00 BOB. No regenerar los datos para esta demostración.

El lote tiene una condición de cierre explícita y archivos inmutables. No ejecutar jobs simultáneos ni mezclar landing BASE y DELTA.

Verificar resultados y recuperar fallos

```
python 02_Codigo/pruebas/test_business.py
python 02_Codigo/pruebas/verify_database.py
--expected data/source/expected.json --expected-phase after_delta
--require-idempotent --output data/evidence/verificacion.json
```

La prueba del sistema

test_transactions.py y test_spark_pipeline.py crean bases aisladas para comprobar rollback y Spark/JDBC. test_dashboard.py prueba filtros y navegación; verify_integration.py consulta datos reales publicados. API verificar_api.py comprueba determinismo y contrato. Los comandos y argumentos completos aparecen en los README de cada componente.

Rendimiento y backup

```
python 02_Codigo/procesamiento/benchmark.py
--output data/evidence/benchmark
python 02_Codigo/almacenamiento/backup_restore.py
--pg-bin C:/ruta/pgsql/bin
```

Situación	Acción
API no disponible	Revisar /health y log; NiFi conserva los fallos/reintentos.
CSV o evento inválido	Consultar originales y motivos en data/quality/ingesta.
Spark falla	Revisar pipeline_runs/log; no cambiar hechos manualmente. Corregir y repetir lote sellado.
Falta un antecedente	Revisar pending_event; aportar el evento previo en un lote posterior.
Panel sin datos	Comprobar conexión y estado del último job; no cargar cifras ficticias.
Restauración	Restaurar en base nueva; comparar SHA256/conteos/cifras antes de usarla.

La carpeta data/evidence conserva resultados de pruebas y huellas SHA de los archivos correspondientes. Los rechazos esperados son parte de la demostración de calidad; no representan una falla que deba ocultarse.